

RGMC 2026 Human-to-Robot Handover

Technical Report

Enrico Turco¹, Chiara Castellani¹, Gionata Salvietti^{1,2}, and Maria Pozzi^{1,2}

¹Istituto Italiano di Tecnologia, Genoa, Italy. {name.surname}@iit.it

²Universita degli Studi di Siena, Dip. di Ingegneria dell'Informazione e Scienze Matematiche, Siena, Italy.

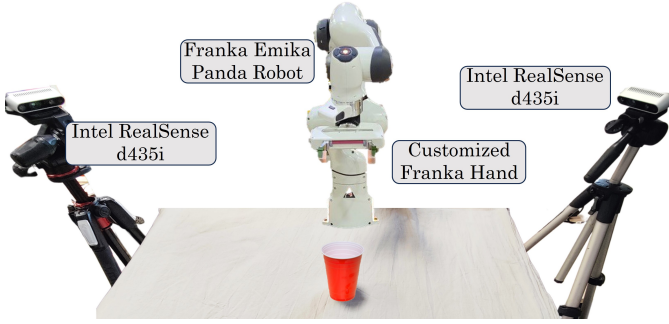


Fig. 1. Experimental setup used by the vision-based handover pipeline. The robot observes the workspace through two external RGB-D cameras placed on opposite sides of the interaction area.

SYSTEM OVERVIEW

This report summarizes the human-to-robot handover system developed by Team SIRSIT for the RGMC handover task. The solution is a fast, markerless, vision-based pipeline that estimates the human hand pose and the object geometry from two external RGB-D streams, generates a grasp pose online, and executes the handover with an event-driven Cartesian velocity controller. The same architecture was used in [1] and was designed to remain independent of object-specific models, fixed handover poses, or a specific robot platform.

The system combines multi-view RGB-D perception with a reactive control strategy designed to remain robust when the object becomes partially occluded during the handover.

HARDWARE AND EXPERIMENTAL SETUP

Table I summarizes the hardware configuration. The main setup consists of a Franka Emika Panda or Franka Research 3 (FR3) collaborative robot arm placed behind a table, a Franka parallel gripper equipped with a 90-degree wrist adapter and fingertip extensions, and two external RGB-D cameras observing the handover area from complementary viewpoints (as shown in Fig. 1). The cameras are rigidly mounted and calibrated with respect to a common world frame attached to the robot/table setup. The human operator stands in front of the table and hands the object to the robot.

The interaction workspace is the table region jointly visible from the two cameras and reachable by the robot. Workspace limits are used both for safety and for perception filtering: hand detections and segmented object points outside this region are discarded. The target placement area is on the table, and the

TABLE I
HARDWARE CONFIGURATION USED FOR THE RGMC 2026.

Component	Configuration
Robot arm	Franka Emika Panda or Franka Research 3 (FR3), 7 DoF
End-effector	Franka parallel gripper with 90-degree wrist adapter and fingertip extensions
Perception	Two Intel RealSense D435i RGB-D cameras
Compute	ASUS ROG Strix G17 and MSI Pulse GL66 laptops running the perception and robot control stack
Robot software	ROS-based architecture with MoveIt Servo for real-time Cartesian velocity commands

task is considered successful when the robot grasps the object, transports it, and releases it stably inside the target area.

SOFTWARE ARCHITECTURE

The software stack is modular and ROS-based. It is composed of perception nodes, a fusion layer, a grasp-generation module, and a hierarchical state machine coordinating the robot behavior. The perception pipeline is shown in Fig. 2.

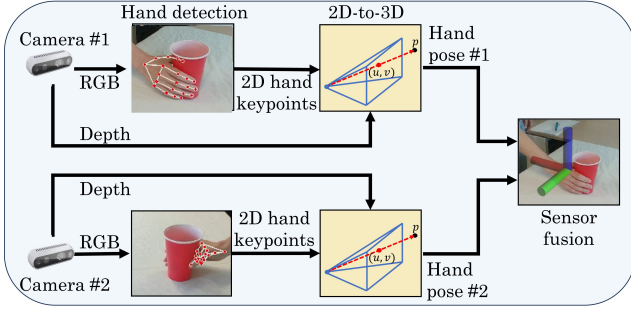
Hand Tracking

Each RGB image is processed with MediaPipe Hands [2]. For each detected hand, the palm-center pixel is deprojected into 3D using the aligned depth image and camera intrinsics. The resulting hand pose is transformed to the world frame using the camera extrinsics. Detections outside the allowed workspace are removed, and if more than one valid hand is detected, the one closest to the robot is selected. This module requires the user's hand to remain visible from at least one RGB-D camera; temporary occlusions and fast motions are mitigated by the two-view setup and by temporal filtering, but they can still degrade the estimate.

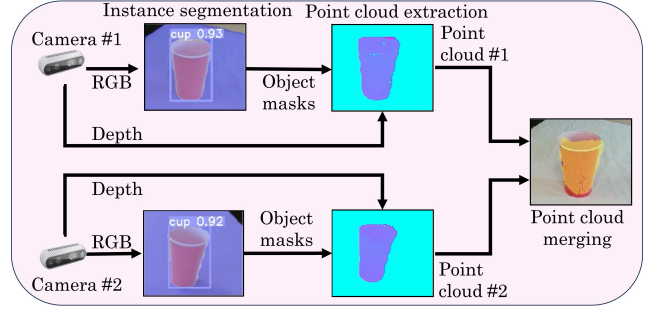
Hand position estimates from the two cameras are asynchronously fused with a six-dimensional linear Kalman filter whose state contains hand position and velocity. Hand orientation is estimated from the 3D keypoints by defining a palm frame, and orientation estimates are smoothed with quaternion spherical linear interpolation. This filtered hand pose is the main feedback signal used during the approach phase.

Object Detection and Geometry Estimation

Object perception uses a YOLOv26 instance-segmentation model [3]. The model is used to obtain binary masks in the



(a) Hand pose estimation from RGB-D streams.



(b) Object reconstruction from segmentation masks and depth.

Fig. 2. Vision-based software pipeline. Hand pose estimation and object reconstruction run independently on each camera stream and are fused in a common world frame.

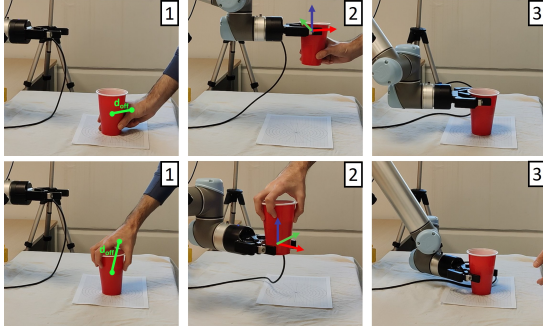


Fig. 3. Representative handover sequences for bottom and top grasps: initial hand-object offset estimation, robot approach and grasp, and stable placement.

RGB image. Masks associated with the human body or classes that commonly generate false positives are discarded, while the remaining masks are merged into a single object mask. The mask filters the depth image, which is then deprojected into a point cloud.

For each camera, the point cloud is cropped to the workspace, voxel-downsampled, and cleaned through outlier removal. The camera-specific clouds are transformed to the world frame and merged. The system then computes an axis-aligned bounding box with Open3D [4]. The bounding box center and dimensions provide a compact, model-free object representation for fast grasp planning.

The object pipeline assumes that enough valid depth points are visible before the object is lifted and that the camera extrinsics remain fixed during the run.

Robot Control and State Machine

The robot starts from a predefined home configuration while the perception pipeline continuously publishes hand pose and object bounding-box estimates. When the hand approaches the object, the controller stores the initial offset

$$\mathbf{d}_{\text{off}}(t_0) = \mathbf{p}_{\text{obj}}(t_0) - \mathbf{p}_{\text{hand}}(t_0), \quad (1)$$

where $\mathbf{p}_{\text{obj}}$ is the object bounding-box center and $\mathbf{p}_{\text{hand}}$ is the filtered palm position.

During the handover, the object can become partially occluded by the user's hand. The controller therefore updates the object estimate from the hand pose. The relative hand yaw is extracted from the hand orientation and used to rotate the initial hand-object offset:

$$\mathbf{d}_{\text{off}}(t) = \mathbf{R}_z(\Delta\psi(t))\mathbf{d}_{\text{off}}(t_0). \quad (2)$$

This initial hand-object offset is a central design element of the controller. The object geometry is reconstructed only while the object is sufficiently visible on the table; after the user lifts it, the controller relies primarily on the tracked hand pose and on the updated offset to infer the object position. This makes the system less sensitive to hand-induced occlusions during the handover. Pitch and roll are ignored because the expected handover motion keeps the object approximately upright, while yaw changes significantly with the user's lateral motion. This yaw-only update is the main geometric assumption of the controller and can become inaccurate if the user strongly tilts the object during the handover.

When a lifting motion is detected, the controller generates a time-varying grasp pose. The grasp position is obtained from the estimated object center and shifted by half of the bounding-box width along the gripper approach direction. The vertical component is adjusted by a fixed fraction of the object height to avoid colliding with the user's fingers. The grasp orientation is selected so that the gripper approaches the object in the horizontal plane while aligning its closing direction with the selected side of the bounding box.

The end-effector tracks the desired grasp pose through Cartesian velocity commands generated with a proportional-derivative controller and sent through MoveIt Servo [5]. The gripper closes when the gripper-object distance falls below the threshold $d_{\text{thr}} = 50$ mm, selected as a compromise between execution time, placement accuracy, and grasp success. Since no tactile sensing is used, the system does not directly estimate contact quality or incipient slip after closure. The robot then transports the object to the estimated initial object location, places it on the table, opens the gripper, and returns to the home configuration. Representative bottom and top grasp sequences are shown in Fig. 3.

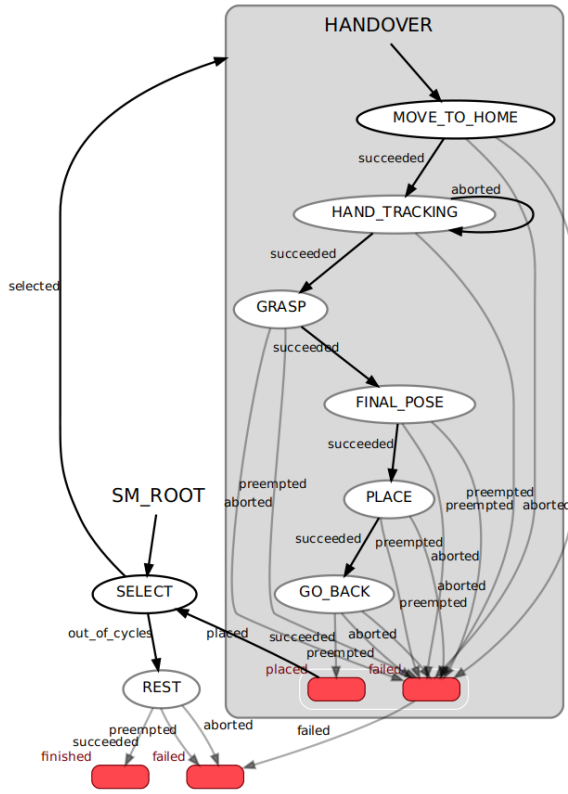


Fig. 4. SMACH handover state machine which coordinates the handover behavior from homing to grasping, placement, and recovery.

At the behavior level, a SMACH state machine [6] coordinates the cycle through the states `MOVE_TO_HOME`, `HAND_TRACKING`, `GRASP`, `FINAL_POSE`, `PLACE`, and `GO_BACK`. The root layer manages the transition between the idle/resting condition and the execution of a handover trial, while the `HANDOVER` container implements the complete operational sequence. This structure keeps perception, grasp generation, robot motion, and recovery logic separated. The state machine structure is depicted in Fig. 4.

PIPELINE VALIDATION

In a preliminary validation, the same core pipeline was tested on a UR5 robot from Universal Robots, equipped with a Robotiq 2F-85 gripper, following the CORSMAL/RGMC handover protocol [7], [8]. The evaluation involved four users, four drinking containers, two filling conditions, three handover locations, and three grasp types. The system completed 216/288 handovers, with an average execution time of 3.25 ± 0.48 s, placement error of 22 ± 16 mm, and negligible spillage. Additional tests on a tennis ball, bowl, and box reached 26/27 successful handovers, supporting the generality of the perception and grasp-planning pipeline beyond the benchmark containers. The main failure modes observed across these validations were related to transparent objects, partial occlusions, and rapid user motions.

REFERENCES

- [1] E. Turco, C. Castellani, G. Salvietti, and M. Pozzi, "Fast human-robot handover using a vision-based pipeline," *IEEE Robotics and Automation Practice*, vol. 1, pp. 66–70, 2026.
- [2] A. Vakunov, C.-L. Chang, F. Zhang, G. Sung, M. Grundmann, and V. Bazarevsky, "MediaPipe Hands: On-device real-time hand tracking," in *Workshop on Computer Vision for AR/VR*, vol. 2, no. 4, 2020, p. 5.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," *arXiv:1801.09847*, 2018.
- [5] B. O'Neil, A. Zelenak, B. Anderson, A. Rössler, T. Weaver, and A. Pettinger, "Realtime arm servoing – moveit tutorials," https://moveit.github.io/moveit_tutorials/doc/realtime_servo/realtime_servo_tutorial.html, 2016, accessed: Apr. 11, 2025.
- [6] J. Bohren and S. Cousins, "The smach high-level executive [ros news]," *IEEE Robotics & Automation Magazine*, vol. 17, no. 4, pp. 18–20, 2010.
- [7] R. Sanchez-Matilla, K. Chatzilygeroudis, A. Modas, N. F. Duarte, A. Xompero, P. Frossard, A. Billard, and A. Cavallaro, "Benchmark for human-to-robot handovers of unseen containers with unknown filling," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1642–1649, 2020.
- [8] A. Xompero, S. Donaher, V. Iashin, F. Palermo, G. Solak, C. Coppola, R. Ishikawa, Y. Nagao, R. Hachiuma, Q. Liu, *et al.*, "The corsmal benchmark for the prediction of the properties of containers," *IEEE Access*, vol. 10, pp. 41 388–41 402, 2022.